Today we will learn

1. Prop Drilling
2. Context API
3. createContext()
4. Provider
5. useContext()
6. useReducer()
7. Actions and dispatch

Today's Goal

By the end of the session, students should understand this flow:

```
Component
    ↓
Context
    ↓
Provider
    ↓
useContext()
    ↓
Shared State
```

And for complex state:

```
Component
    ↓
dispatch(action)
    ↓
Reducer
    ↓
New State
    ↓
UI Updates
```

Slide 2. Prop Drilling

First understand the problem

Imagine we have:

```
App
 ↓
Navbar
```

```
 ↓
Profile
 ↓
User
```

We have user information in `App`.

```
const user = {
  name: "Rahul",
  role: "Developer"
};
```

But `Profile` needs the user.

So we pass it through every component.

App.jsx

```
function App() {

  const user = {
    name: "Rahul",
    role: "Developer"
  };

  return <Navbar user={user} />;
}
```

Navbar.jsx

```
function Navbar({ user }) {

  return <Profile user={user} />;

}
```

Profile.jsx

```
function Profile({ user }) {

  return (
    <h2>
      Welcome {user.name}
    </h2>
  );

}
```

What is the problem?

`Navbar` does not actually need `user`.

It is only passing the data to `Profile`.

This is called **Prop Drilling**.

Why avoid it?

As applications become larger:

```
App
 ↓
Navbar
 ↓
Layout
 ↓
Sidebar
 ↓
Profile
 ↓
User
```

Passing props through every level becomes difficult to maintain.

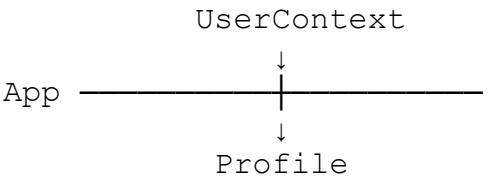
Slide 3. Context API, Create Our First Context

Context API solves Prop Drilling

Instead of:

```
App → Navbar → Profile
```

We can do:



Step 1, Create Context

Create:

```
src/context/UserContext.jsx
```

Code:

```
import { createContext } from "react";

const UserContext = createContext();

export default UserContext;
```

Step by Step:

`createContext()`

Creates a Context object.

Think of it as creating a shared data channel.

```
UserContext
    ↓
Shared User Data
```

Why export it?

Other components need to access the same context.

```
export default UserContext;
```

Slide 4. Provider, Make Data Available

Creating Context alone is not enough.

We need to provide data to components.

This is where **Provider** comes in.

App.jsx

```
import UserContext from "./context/UserContext";

function App() {

  const user = {
    name: "Rahul",
    role: "Developer"
  };

  return (

    <UserContext.Provider value= {user}>

      <Profile />
```

```
    </UserContext.Provider>

  );
}

export default App;
```

Understand the Code

Step 1

We create user data.

```
const user = {
  name: "Rahul",
  role: "Developer"
};
```

Step 2

We use Provider.

```
<UserContext.Provider>
```

Step 3

We provide the data.

```
value={user}
```

Step 4

Any component inside Provider can access this data.

```
UserContext.Provider
        ↓
      Profile
        ↓
Can access user
```

Important Rule

A component must be **inside the Provider** to access the context value.

Slide 5. useContext(), Access the Data

Now `Profile` needs the user information.

Profile.jsx

```
import { useContext } from "react";
import UserContext from "../context/UserContext";

function Profile() {

  const user = useContext(UserContext);

  return (
    <div>
      <h2>{user.name}</h2>
      <p>{user.role}</p>
    </div>
  );
}

export default Profile;
```

Step by Step

Step 1

Import `useContext`.

```
import { useContext } from "react";
```

Step 2

Import our context.

```
import UserContext from "../context/UserContext";
```

Step 3

Read the context.

```
const user = useContext(UserContext);
```

Now:

```
user.name
```

returns:

```
Rahul
```

And:

```
user.role
```

returns:

```
Developer
```

Complete Flow

```
App
 ↓
Provider
 ↓
value={user}
 ↓
Profile
 ↓
useContext(UserContext)
 ↓
user
```

No props required.

Slide 6. useReducer(), Why Do We Need It?

`useState()` works very well for simple state.

Example:

```
const [count, setCount] = useState(0);
```

But imagine a shopping cart.

We may need:

```
ADD_TO_CART
REMOVE_FROM_CART
INCREASE_QUANTITY
DECREASE_QUANTITY
CLEAR_CART
```

Managing all these state changes with many `setState()` calls can become difficult.

This is where `useReducer()` helps.

Basic Syntax

```
const [state, dispatch] = useReducer(
  reducer,
  initialState
);
```

There are three important things.

```
state
```

```
 ↓
Current Data

dispatch
 ↓
Send Action

reducer
 ↓
Decides New State
```

Slide 7. Building useReducer Step by Step

Let's create a simple counter.

Step 1, Initial State

```
const initialState = {
  count: 0
};
```

Our initial state is:

```
count = 0
```

Step 2, Create Reducer

```
function reducer(state, action) {

  switch (action.type) {

    case "INCREMENT":
      return {
        count: state.count + 1
      };

    case "DECREMENT":
      return {
        count: state.count - 1
      };

    default:
      return state;
  }
}
```

Understand the Reducer

The reducer receives two things.

```
reducer(state, action)
```

`state` contains the current data.

`action` tells us what should happen.

For example:

```
{
  type: "INCREMENT"
}
```

Reducer sees:

```
action.type
```

and executes:

```
case "INCREMENT":
```

Slide 8. dispatch(), Connecting the UI to Reducer

Now create the reducer state.

```
import { useReducer } from "react";

const [state, dispatch] = useReducer(
  reducer,
  initialState
);
```

Display the value:

```
<h2>{state.count}</h2>
```

Increment button:

```
<button
  onClick={() =>
    dispatch({ type: "INCREMENT" })
  }
>
  +

</button>
```

Decrement:

```
<button
  onClick={() =>
    dispatch({
      type: "DECREMENT"
    })
  }
>
  -
</button>
```

Understand the Flow

When user clicks +:

```
User clicks +
      ↓
dispatch()
      ↓
{ type: "INCREMENT" }
      ↓
reducer()
      ↓
case "INCREMENT"
      ↓
count + 1
      ↓
New State
      ↓
React updates UI
```

This is the most important flow students should understand.

Slide 9. Context API + useReducer

Now the task is combine both concepts [Add to cart Feature]

Slide 10. Mini Project and Homework

Mini Project, Theme Switcher

Build a Theme Switcher using Context API.

Features

```
Light Mode
Dark Mode
Toggle Button
```

Create:

```
context/
    ThemeContext.jsx

components/
    Header.jsx
    ThemeButton.jsx
```

Expected Flow

```
ThemeProvider
      ↓
Theme State
      ↓
useContext()
      ↓
Header
      ↓
Toggle Theme
```

Homework, Authentication Context

Build an Authentication system using **Context API + useReducer**.

State

```
const initialState = {
  isLoggedIn: false,
  user: null
};
```

Actions

```
LOGIN
LOGOUT
```

Login

```
dispatch({
  type: "LOGIN",
  payload: {
    name: "Rahul",
    email: "rahul@example.com"
  }
});
```

Logout

```
dispatch({
  type: "LOGOUT"
```

```
});
```

Requirements

1. Create `AuthContext`.
2. Create `AuthProvider`.
3. Use `useReducer()`.
4. Implement LOGIN.
5. Implement LOGOUT.
6. Use `useContext()` inside Header.
7. Show username when logged in.
8. Show Login button when logged out.
9. Show Logout button when logged in.
10. Display a protected Dashboard message.

Expected UI

Logged Out:

```
Welcome Guest

[ Login ]
```

Logged In:

```
Welcome Rahul

[ Logout ]

Dashboard
```

Student Challenge

Add React Router protection.

If the user is not logged in and tries to access:

```
/dashboard
```

redirect them to:

```
/login
```

This gives students a practical introduction to how **Context API, useReducer, and React Router work together in a real application**.